

SLOT4:

Stylesheets, Styling with Layouts and Flexbox, Using Styled-Components



Tóm tắt nội dung

1. Introduction to Stylesheets (Giới thiệu về Stylesheets)

- **Mục tiêu:** Hiểu cách áp dụng CSS trong React Native.
- **Nội dung chính:**
 - React Native không dùng CSS trực tiếp mà sử dụng `StyleSheet.create()` để định nghĩa kiểu.
 - Các thuộc tính tương tự CSS nhưng được viết theo cú pháp camelCase (ví dụ: `backgroundColor`, `fontSize`).
 - Style được áp dụng thông qua `style={styles.tênStyle}`.

2. Styling with Layouts and Flexbox (Dàn trang và bố cục bằng Flexbox)

- **Mục tiêu:** Biết cách dàn trang các thành phần bằng hệ thống Flexbox.
- **Nội dung chính:**
 - React Native dùng Flexbox để xây bố cục (layout), mặc định là hướng dọc (`flexDirection: 'column'`).
 - Các thuộc tính quan trọng:
 - `flex`, `flexDirection`, `justifyContent`, `alignItems`
 - Dùng để căn chỉnh, giãn đều, canh giữa hoặc sắp xếp các thành phần.
 - Giúp tạo giao diện phản hồi tốt trên nhiều kích thước màn hình.

3. Using Styled-Components in React Native

- **Mục tiêu:** Làm quen với thư viện `styled-components` để viết style theo cách component hóa.
- **Nội dung chính:**
 - Thay vì dùng `StyleSheet`, bạn viết CSS trực tiếp trong component bằng cú pháp template literals.
 - Ví dụ:

```
jsx

const StyledView = styled.View`
  background-color: papayawhip;
  padding: 10px;
`;
```

- Giúp style rõ ràng hơn, dễ tái sử dụng, và kết hợp tốt với logic JS.

✓ CÁC NỘI DUNG TRỌNG TÂM, CỐT LÕI CẦN LƯU Ý:

✂ 1. Introduction to Stylesheets

◆ Cốt lõi:

- React Native **không dùng CSS truyền thống**, thay vào đó dùng `StyleSheet.create()`.
- Style viết theo **kiểu đối tượng JS**, sử dụng **camelCase** cho thuộc tính (ví dụ: `fontSize`, `backgroundColor`).
- Có thể gán nhiều style bằng **mảng**: `style=[styles.base, styles.highlight]`.

◆ Ghi nhớ:

- Style trong React Native là **cục bộ**, không có kế thừa như CSS.
- Không hỗ trợ toàn bộ thuộc tính CSS (ví dụ: `box-shadow`, `z-index` bị hạn chế).

✂ 2. Styling with Layouts and Flexbox

◆ Cốt lõi:

- React Native dùng **Flexbox mặc định theo chiều dọc** (`flexDirection: 'column'`).
- Các thuộc tính Flexbox hay dùng:
 - `flex`: chiếm không gian theo tỷ lệ
 - `flexDirection`: 'row' hoặc 'column'
 - `justifyContent`: căn chỉnh theo trục chính
 - `alignItems`: căn chỉnh theo trục phụ

◆ Ghi nhớ:

- Dùng Flexbox để tạo bố cục linh hoạt, phản hồi tốt trên nhiều thiết bị.
- Việc hiểu **trục chính** và **trục phụ** là chìa khóa để làm chủ layout.

✂ 3. Using Styled-Components in React Native

◆ Cốt lõi:

- `styled-components` giúp viết style **dễ đọc, dễ tái sử dụng**, theo dạng component.
- Viết style như CSS trong JS với cú pháp **template literal** (`` ``).
- Kết hợp dễ dàng với props để thay đổi style động.

◆ Ghi nhớ:

- Cần cài thư viện: `npm install styled-components`
- Có thể tách style riêng thành **các component tái sử dụng**.
- Giữ mã ngắn gọn và dễ bảo trì hơn khi dự án phức tạp.

👉 Tóm gọn:

- React Native dùng JS để viết style, không phải CSS thuần.
- Flexbox là công cụ bố cục chính, rất quan trọng.

- styled-components là lựa chọn nâng cao giúp viết style đẹp và tổ chức tốt hơn

✓ **ỨNG DỤNG TRONG THỰC TIỄN:**

✓ 1. Introduction to Stylesheets

Ứng dụng thực tiễn:

- Tạo **giao diện cơ bản**: màu nền, màu chữ, cỡ chữ, viền, padding, margin...
- Xây dựng **bộ style tái sử dụng**: ví dụ như button, title, container cho toàn bộ app.
- Áp dụng logic hiển thị theo điều kiện:

```
jsx
```

```
<Text style={isError ? styles.errorText : styles.normalText}>...</Text>
```

✓ 2. Styling with Layouts and Flexbox

Ứng dụng thực tiễn:

- Dàn layout các thành phần trên màn hình:
 - Ví dụ: dàn nút theo hàng ngang, căn giữa avatar, chia cột ảnh và nội dung...
- Đảm bảo **giao diện hiển thị tốt trên nhiều kích thước màn hình** (responsive).
- Thiết kế **UI động** như bottom tab, header cố định, card responsive, v.v.
- Tạo bố cục như:

```
jsx
```

```
<View style={{ flexDirection: 'row', justifyContent: 'space-between'
}}>
  <Button title="Back" />
  <Button title="Next" />
</View>
```

✓ 3. Using Styled-Components in React Native

Ứng dụng thực tiễn:

- Tạo **giao diện sạch, dễ đọc** khi làm việc nhóm hoặc dự án lớn.
- Tách UI thành các **component nhỏ với style riêng**:

```
jsx
```

```
const Card = styled.View`
  background-color: white;
  padding: 16px;
  border-radius: 10px;
`;
```

- Dễ **điều chỉnh giao diện theo logic hoặc props**:

```
jsx
```

```
const Title = styled.Text`
  color: ${props => props.highlight ? 'red' : 'black'};
`;
```

Tổng kết ứng dụng thực tế

Kỹ năng	Ứng dụng thực tế
StyleSheet	Tạo giao diện cơ bản, tái sử dụng style
Flexbox Layout	Thiết kế bố cục linh hoạt cho mọi loại màn hình
Styled-Components	Viết UI rõ ràng, có cấu trúc tốt, dễ bảo trì và mở rộng

CHI TIẾT MỘT SỐ NỘI DUNG:

1. StyleSheet trong React Native

Khái niệm cơ bản

Trong React Native, việc định nghĩa và áp dụng kiểu dáng (styles) cho các thành phần giao diện được thực hiện thông qua JavaScript, thay vì sử dụng CSS thuần túy như trên web. React Native cung cấp một API có tên `StyleSheet` để tạo và quản lý các kiểu dáng một cách hiệu quả.

Cách sử dụng `StyleSheet.create()`

```
javascript
Sao chépChỉnh sửa
import { StyleSheet } from 'react-native';

const styles = StyleSheet.create({
  container: {
    backgroundColor: '#fff',
    padding: 10,
  },
  title: {
    fontSize: 20,
    fontWeight: 'bold',
  },
});
```

- **`StyleSheet.create()`**: Tạo một đối tượng chứa các kiểu dáng được định nghĩa. Việc sử dụng phương thức này giúp React Native thực hiện kiểm tra kiểu tĩnh và tối ưu hóa hiệu suất.
- **Cú pháp camelCase**: Các thuộc tính kiểu dáng sử dụng cú pháp camelCase thay vì kebab-case như trong CSS. Ví dụ: `backgroundColor` thay vì `background-color`.
- **Áp dụng kiểu dáng**: Sử dụng prop `style` để áp dụng kiểu dáng cho các thành phần:

```
javascript
Sao chépChỉnh sửa
<View style={styles.container}>
  <Text style={styles.title}>Chào mừng!</Text>
</View>
```

Lưu ý quan trọng

- **Không có kế thừa kiểu dáng**: Khác với CSS, React Native không hỗ trợ kế thừa kiểu dáng giữa các thành phần.

- **Kết hợp nhiều kiểu dáng:** Có thể kết hợp nhiều kiểu dáng bằng cách truyền một mảng vào prop `style`. Kiểu dáng ở vị trí sau sẽ ghi đè các thuộc tính trùng lặp của kiểu dáng ở vị trí trước:

```
javascript
Sao chépChỉnh sửa
<Text style={[styles.baseText, styles.highlightText]}>Nội dung</Text>
```

2. Bố cục với Flexbox

Khái niệm cơ bản

React Native sử dụng hệ thống bố cục Flexbox để sắp xếp các thành phần trong giao diện. Flexbox cung cấp một cách linh hoạt để xác định cách các thành phần con được phân bố trong một thành phần cha, cả theo chiều ngang và chiều dọc.

Các thuộc tính Flexbox quan trọng

- **flexDirection:** Xác định hướng chính của bố cục. Giá trị mặc định là `'column'`.

```
javascript
Sao chépChỉnh sửa
flexDirection: 'row' // hoặc 'column'
```

- **justifyContent:** Căn chỉnh các thành phần con theo hướng chính.

```
javascript
Sao chépChỉnh sửa
justifyContent: 'center' // hoặc 'flex-start', 'flex-end', 'space-between', 'space-around', 'space-evenly'
```

- **alignItems:** Căn chỉnh các thành phần con theo hướng phụ (vuông góc với hướng chính).

```
javascript
Sao chépChỉnh sửa
alignItems: 'center' // hoặc 'flex-start', 'flex-end', 'stretch', 'baseline'
```

- **flex:** Xác định tỷ lệ không gian mà một thành phần con chiếm trong thành phần cha.

```
javascript
Sao chépChỉnh sửa
flex: 1 // Thành phần sẽ chiếm toàn bộ không gian còn lại
```

Lưu ý quan trọng

- **Hướng mặc định:** Trong React Native, hướng mặc định của Flexbox là `'column'`, tức là các thành phần con được sắp xếp theo chiều dọc.
- **Bố cục phản hồi (responsive):** Flexbox giúp tạo ra các bố cục linh hoạt, tự điều chỉnh phù hợp với nhiều kích thước màn hình khác nhau.
- **Kết hợp các thuộc tính:** Việc kết hợp `flexDirection`, `justifyContent`, và `alignItems` một cách hợp lý sẽ giúp bạn xây dựng các bố cục phức tạp một cách dễ dàng.

3. Sử dụng Styled-Components trong React Native

Khái niệm cơ bản

`styled-components` là một thư viện cho phép bạn viết kiểu dáng cho các thành phần React Native bằng cách sử dụng cú pháp template literals của JavaScript. Thay vì định nghĩa kiểu dáng trong một đối tượng riêng biệt, bạn có thể gắn kiểu dáng trực tiếp vào thành phần, giúp mã nguồn trở nên rõ ràng và dễ bảo trì hơn.

Cách sử dụng `styled-components`

1. Cài đặt thư viện:

```
bash
Sao chépChỉnh sửa
npm install styled-components
```

2. Định nghĩa thành phần với kiểu dáng:

```
javascript
Sao chépChỉnh sửa
import styled from 'styled-components/native';

const StyledView = styled.View`
  background-color: papayawhip;
  padding: 10px;
`;

const StyledText = styled.Text`
  color: palevioletred;
  font-size: 20px;
`;
```

3. Sử dụng trong component:

```
javascript
Sao chépChỉnh sửa
const App = () => (
  <StyledView>
    <StyledText>Chào mừng đến với ứng dụng!</StyledText>
  </StyledView>
);
```

Lưu ý quan trọng

- **Tái sử dụng và tổ chức mã nguồn:** `styled-components` giúp bạn tái sử dụng kiểu dáng và tổ chức mã nguồn một cách hiệu quả, đặc biệt trong các dự án lớn.
- **Kiểu dáng động:** Bạn có thể thay đổi kiểu dáng dựa trên props:

```
javascript
Sao chépChỉnh sửa
const StyledText = styled.Text`
  color: ${props => (props.primary ? 'blue' : 'gray')};
`;
```

- **Hỗ trợ đầy đủ các thuộc tính kiểu dáng:** styled-components hỗ trợ hầu hết các thuộc tính kiểu dáng mà React Native cung cấp.

✓ 1. Sử dụng StyleSheet

Trình tự các bước:

1. **Import StyleSheet** từ react-native:

```
js
import { StyleSheet } from 'react-native';
```

2. **Tạo đối tượng style** bằng StyleSheet.create():

```
js
const styles = StyleSheet.create({
  container: {
    backgroundColor: 'white',
    padding: 10,
  },
});
```

3. **Áp dụng style** cho thành phần:

```
js
<View style={styles.container}></View>
```

4. **(Tuỳ chọn):** Kết hợp nhiều style bằng mảng:

```
js
<Text style={[styles.base, styles.highlight]} />
```

✓ 2. Sử dụng Layout với Flexbox

Trình tự các bước:

1. **Xác định bố cục cha dùng Flexbox:**

```
js
<View style={styles.wrapper}></View>
```

2. **Áp dụng các thuộc tính Flexbox trong style:**

```
js
const styles = StyleSheet.create({
  wrapper: {
    flexDirection: 'row', // hoặc 'column'
    justifyContent: 'center', // sắp xếp theo trục chính
    alignItems: 'center', // sắp xếp theo trục phụ
  }
});
```

```
    },
  });
```

3. Dùng `flex` cho các phần tử con nếu cần chia tỷ lệ không gian:

```
js

<View style={{ flex: 1 }} />
<View style={{ flex: 2 }} />
```

✓ 3. Sử dụng `Styled-Components`

Trình tự các bước:

1. Cài đặt thư viện:

```
bash

npm install styled-components
```

2. Import `styled` từ `styled-components/native`:

```
js

import styled from 'styled-components/native';
```

3. Tạo component với style gắn kèm:

```
js

const Container = styled.View`
  padding: 20px;
  background-color: #fff;
`;

const Title = styled.Text`
  font-size: 24px;
  color: blue;
`;
```

4. Sử dụng trong `JSX` như component bình thường:

```
js

<Container>
  <Title>Chào mừng</Title>
</Container>
```

5. (Tuỳ chọn): Truyền props để thay đổi style động:

```
js

const Title = styled.Text`
  color: ${props => (props.primary ? 'blue' : 'gray')};
`;

<Title primary>Tiêu đề chính</Title>
```


✂ So sánh ngắn gọn:

Phương pháp	Ưu điểm chính	Sử dụng khi nào
StyleSheet	Nhẹ, nhanh, tích hợp sẵn	Dự án nhỏ, logic đơn giản
Flexbox	Bố cục linh hoạt, responsive tốt	Mọi dự án, xử lý layout động
Styled-Components	Rõ ràng, dễ tái sử dụng, hiện đại	Dự án lớn, code nhiều component UI

🎯 KINH NGHIỆM THỰC TIỄN, VẤN ĐỀ CẦN LƯU Ý, VÀ CÁC MẸO (TIPS)

✓ 1. Kinh nghiệm thực tiễn

◆ StyleSheet

- Tách style ra ngoài component để mã rõ ràng hơn.
- Tái sử dụng style giống nhau trong nhiều màn hình bằng cách **đặt style chung trong 1 file riêng** (ví dụ: `styles/common.js`).
- Dùng biến toàn cục cho màu sắc, kích thước (theme constants) để đồng bộ giao diện.

◆ Flexbox

- Luôn nhớ: **trục chính mặc định là dọc** (`flexDirection: 'column'`) → dễ gây nhầm lẫn nếu quen với CSS trên web.
- Dùng `flex: 1` cho **container cấp cao nhất** để đảm bảo nó chiếm toàn bộ màn hình (đặc biệt khi dùng `SafeAreaView`).
- Khi bố cục phức tạp: hãy chia nhỏ thành nhiều `View` lồng nhau → dễ kiểm soát hơn.

◆ Styled-Components

- Dùng styled-components để xây dựng **design system**: Button, Input, Card... → rất dễ bảo trì khi dự án lớn.
- Tận dụng props để tạo style động, ví dụ: `<Button primary />` thay đổi màu nền và màu chữ.
- Có thể kết hợp styled-components + `StyleSheet.create()` nếu muốn tối ưu hiệu suất ở các thành phần nhỏ.

⚠ 2. Vấn đề thường gặp & cần lưu ý

Vấn đề / sai lầm	Cách khắc phục
Quên <code>flex: 1</code> trong container	Màn hình không hiển thị đúng → luôn kiểm tra layout bao ngoài cùng
Dùng sai tên thuộc tính style	React Native dùng camelCase (ví dụ: <code>backgroundColor</code> , không phải <code>background-color</code>)
Layout không giãn đúng	Cần xem lại <code>flexDirection</code> , <code>justifyContent</code> , <code>alignItems</code>
Lỗi style bị ghi đè	Cẩn thận khi kết hợp nhiều style bằng mảng <code>[style1, style2]</code>
Dùng styled-components sai cách	Đừng lồng quá nhiều component styled → chia nhỏ logic cho dễ bảo trì

💡 3. Tips & Mẹo hữu ích

🧠 Styling hiệu quả:

- Dùng **const** để **định nghĩa màu sắc** (primaryColor, secondaryColor...) để dễ chỉnh sửa khi đổi theme.
- Dùng thư viện hỗ trợ như:
 - react-native-size-matters → scale kích thước phù hợp nhiều màn hình.
 - react-native-responsive-screen → xử lý theo % màn hình.

✂️ Flexbox nhanh gọn:

- alignItems: 'center' + justifyContent: 'center' → căn giữa toàn bộ nội dung.
- space-between hoặc space-around rất hữu ích khi sắp xếp các nút ở footer hoặc header.

🌟 Styled-Components:

- Tạo thư mục /components/styled/ để chứa các styled component riêng biệt.
- Dùng kiểu đặt tên nhất quán: StyledButton, StyledText, Container, Wrapper, v.v.

🔗 Kết luận

Học gì	Lưu ý gì
StyleSheet	Sạch, nhanh, nhưng có thể rối nếu style phức tạp
Flexbox	Cần hiểu rõ trục chính/trục phụ để layout đúng
Styled-Components	Rất mạnh cho dự án lớn, cần quản lý file hợp lý

✅ 1. Kinh nghiệm thực tiễn

◆ StyleSheet

- **Tách style ra ngoài** component để mã rõ ràng hơn.
- Tái sử dụng style giống nhau trong nhiều màn hình bằng cách **đặt style chung trong 1 file riêng** (ví dụ: styles/common.js).
- Dùng biến toàn cục cho màu sắc, kích thước (theme constants) để đồng bộ giao diện.

◆ Flexbox

- Luôn nhớ: **trục chính mặc định là dọc** (flexDirection: 'column') → dễ gây nhầm lẫn nếu quen với CSS trên web.
- Dùng flex: 1 cho **container cấp cao nhất** để đảm bảo nó chiếm toàn bộ màn hình (đặc biệt khi dùng SafeAreaView).
- Khi bố cục phức tạp: hãy chia nhỏ thành nhiều View lồng nhau → dễ kiểm soát hơn.

◆ Styled-Components

- Dùng styled-components để xây dựng **design system**: Button, Input, Card... → rất dễ bảo trì khi dự án lớn.
- Tận dụng props để tạo style động, ví dụ: <Button primary /> thay đổi màu nền và màu chữ.

- Có thể kết hợp styled-components + `StyleSheet.create()` nếu muốn tối ưu hiệu suất ở các thành phần nhỏ.

⚠ 2. Vấn đề thường gặp & cần lưu ý

Vấn đề / sai lầm	Cách khắc phục
Quên <code>flex: 1</code> trong container	Màn hình không hiển thị đúng → luôn kiểm tra layout bao ngoài cùng
Dùng sai tên thuộc tính style	React Native dùng camelCase (ví dụ: <code>backgroundColor</code> , không phải <code>background-color</code>)
Layout không giãn đúng	Cần xem lại <code>flexDirection</code> , <code>justifyContent</code> , <code>alignItems</code>
Lỗi style bị ghi đè	Cẩn thận khi kết hợp nhiều style bằng mảng <code>[style1, style2]</code>
Dùng styled-components sai cách	Đừng lồng quá nhiều component styled → chia nhỏ logic cho dễ bảo trì

💡 3. Tips & Mẹo hữu ích

🧠 Styling hiệu quả:

- Dùng **const để định nghĩa màu sắc** (`primaryColor`, `secondaryColor...`) để dễ chỉnh sửa khi đổi theme.
- Dùng thư viện hỗ trợ như:
 - `react-native-size-matters` → scale kích thước phù hợp nhiều màn hình.
 - `react-native-responsive-screen` → xử lý theo % màn hình.

✂ Flexbox nhanh gọn:

- `alignItems: 'center' + justifyContent: 'center'` → căn giữa toàn bộ nội dung.
- `space-between` hoặc `space-around` rất hữu ích khi sắp xếp các nút ở footer hoặc header.

🌟 Styled-Components:

- Tạo thư mục `/components/styled/` để chứa các styled component riêng biệt.
- Dùng kiểu đặt tên nhất quán: `StyledButton`, `StyledText`, `Container`, `Wrapper`, v.v.

🔗 Kết luận

Học gì	Lưu ý gì
StyleSheet	Sạch, nhanh, nhưng có thể rối nếu style phức tạp
Flexbox	Cần hiểu rõ trục chính/trục phụ để layout đúng
Styled-Components	Rất mạnh cho dự án lớn, cần quản lý file hợp lý

EXERCISE



TÓM TẮT DỰ ÁN

Tên dự án: MiniApp "Profile Card Viewer"

Mục tiêu học tập:

- ✓ Thành thạo sử dụng `StyleSheet` để định nghĩa style cơ bản
- ✓ Áp dụng `Flexbox` để xây dựng bố cục linh hoạt
- ✓ Thực hành `styled-components` trong React Native
- ✓ Làm quen với cách tổ chức và cấu trúc project thực tế



CHỨC NĂNG CỦA ỨNG DỤNG

1. Hiển thị danh sách profile (tên, avatar, nghề nghiệp)
2. Layout chia cột hàng bằng `Flexbox`
3. Áp dụng style cơ bản bằng `StyleSheet`
4. Một màn hình sử dụng `styled-components`
5. Responsive đơn giản: chạy đẹp trên nhiều kích thước màn hình

CÔNG NGHỆ SỬ DỤNG

Công nghệ	Mục đích
React Native (CLI hoặc Expo)	Xây dựng giao diện ứng dụng di động
StyleSheet	Định nghĩa style tĩnh, cơ bản
Flexbox Layout	Xây dựng bố cục
styled-components	Thực hành style theo component
React Navigation (tuỳ chọn)	Di chuyển giữa các màn hình



CẤU TRÚC THƯ MỤC

bash

```
ProfileCardApp/
├── assets/                # Ảnh avatar mẫu
├── components/           # Các component dùng chung
│   └── ProfileCard.js    # Thẻ profile với style
├── screens/              # Màn hình chính
│   ├── HomeScreen.js
│   └── StyledScreen.js
├── styles/               # StyleSheet dùng chung
│   └── common.js
├── App.js
└── package.json
```

TÓM TẮT DANH SÁCH BƯỚC THỰC HIỆN

1. Khởi tạo dự án React Native
2. Tạo dữ liệu mẫu (profiles)
3. Tạo ProfileCard với StyleSheet
4. Tạo màn hình chính (HomeScreen) dùng Flexbox
5. Thêm StyledScreen sử dụng styled-components
6. Tổ chức style vào thư mục riêng
7. Chạy thử và kiểm tra responsive

CHI TIẾT TỪNG BƯỚC THỰC HIỆN

BƯỚC 1: Khởi tạo dự án

Với Expo (đề xuất cho người mới):

```
bash
```

```
npx create-expo-app ProfileCardApp
cd ProfileCardApp
npm install styled-components
```

Với React Native CLI (tùy chọn):

```
bash
```

```
npx react-native init ProfileCardApp
cd ProfileCardApp
npm install styled-components
```

BƯỚC 2: Tạo dữ liệu mẫu

Tạo file data/profiles.js:

```
js
```

```
export const profiles = [
  {
    id: 1,
    name: 'Nguyễn Văn A',
    job: 'Frontend Developer',
    avatar: require('../assets/avatar1.png'),
  },
  {
    id: 2,
    name: 'Trần Thị B',
    job: 'UX Designer',
    avatar: require('../assets/avatar2.png'),
  },
  // thêm vài profile khác
];
```

BƯỚC 3: Tạo ProfileCard component

Tạo file components/ProfileCard.js:

```
js

import { View, Text, Image, StyleSheet } from 'react-native';

const ProfileCard = ({ name, job, avatar }) => {
  return (
    <View style={styles.card}>
      <Image source={avatar} style={styles.avatar} />
      <View style={styles.info}>
        <Text style={styles.name}>{name}</Text>
        <Text style={styles.job}>{job}</Text>
      </View>
    </View>
  );
};

const styles = StyleSheet.create({
  card: {
    flexDirection: 'row',
    backgroundColor: '#fff',
    padding: 16,
    borderRadius: 10,
    marginVertical: 8,
    alignItems: 'center',
    elevation: 2,
  },
  avatar: {
    width: 60,
    height: 60,
    borderRadius: 30,
    marginRight: 16,
  },
  info: {
    flex: 1,
  },
  name: {
    fontSize: 18,
    fontWeight: 'bold',
  },
  job: {
    fontSize: 14,
    color: 'gray',
  },
});

export default ProfileCard;
```

BƯỚC 4: Tạo màn hình chính (HomeScreen)

Tạo file screens/HomeScreen.js:

```
js

import { View, ScrollView, StyleSheet } from 'react-native';
import ProfileCard from '../components/ProfileCard';
```

```
import { profiles } from '../data/profiles';

const HomeScreen = () => {
  return (
    <ScrollView style={styles.container}>
      {profiles.map(profile => (
        <ProfileCard key={profile.id} {...profile} />
      ))}
    </ScrollView>
  );
};

const styles = StyleSheet.create({
  container: {
    padding: 16,
    backgroundColor: '#f2f2f2',
    flex: 1,
  },
});

export default HomeScreen;
```

BƯỚC 5: Tạo màn hình styledScreen dùng styled-components

Tạo file screens/StyledScreen.js:

```
js

import styled from 'styled-components/native';
import { profiles } from '../data/profiles';

const Container = styled.ScrollView`
  padding: 20px;
  background-color: #e6f0ff;
`;

const Card = styled.View`
  flex-direction: row;
  background-color: white;
  border-radius: 10px;
  margin-bottom: 12px;
  padding: 16px;
  align-items: center;
`;

const Avatar = styled.Image`
  width: 60px;
  height: 60px;
  border-radius: 30px;
  margin-right: 16px;
`;

const Info = styled.View``;

const Name = styled.Text`
  font-size: 18px;
  font-weight: bold;
`;

const Job = styled.Text`
```

```

    font-size: 14px;
    color: gray;
  `;

const StyledScreen = () => (
  <Container>
    {profiles.map(profile => (
      <Card key={profile.id}>
        <Avatar source={profile.avatar} />
        <Info>
          <Name>{profile.name}</Name>
          <Job>{profile.job}</Job>
        </Info>
      </Card>
    ))}
  </Container>
);

export default StyledScreen;

```

✅ BƯỚC 6: Cấu trúc style chung (nếu cần)

Tạo file styles/common.js:

```

js

export const colors = {
  primary: '#4e89ff',
  secondary: '#f2f2f2',
  text: '#333',
};

```

Bạn có thể import và sử dụng trong các style:

```

js

import { colors } from '../styles/common';

color: colors.primary;

```

✅ BƯỚC 7: Chạy thử và kiểm tra

bash

```
npm start # nếu dùng Expo
```

Đảm bảo layout hoạt động tốt trên nhiều kích thước màn hình (dọc/ngang).

📁 GỢI Ý MỞ RỘNG

- Thêm **navigation** giữa HomeScreen và StyledScreen (dùng react-navigation)
- Cho phép click vào thẻ để mở chi tiết profile
- Học thêm về TouchableOpacity, useState, truyền props nâng cao

RÈN LUYỆN

PHÂN TÍCH QUY LUẬT THỰC THI

PROMT: Tôi là người mới học, tôi cần phát hiện quy luật thực thi của chương trình. Hãy Mô tả quá trình thực thi của code của bài tập

PHẦN 1: HƯỚNG DẪN DỰ ÁN NODEJS PHỤC VỤ BÀI 5

TÓM TẮT MỤC TIÊU DỰ ÁN

- Tạo một RESTful API bằng Node.js để cung cấp dữ liệu cho app React Native.
- Hiểu cách tổ chức project Node.js chuẩn.
- Rèn kỹ năng xử lý dữ liệu JSON và phản hồi API.
- Làm nền tảng để thực hành styling ở frontend (React Native).

CÁC CHỨC NĂNG CỦA DỰ ÁN

1. API trả danh sách người dùng (/profiles)
2. API thêm người dùng mới (POST /profiles)
3. API lấy chi tiết theo ID (/profiles/:id)
4. Có thể dễ dàng mở rộng cho việc kết nối React Native

CÔNG NGHỆ SỬ DỤNG

Công nghệ	Mục đích
Node.js	Nền tảng chính
Express.js	Tạo API RESTful
Nodemon	Tự động reload khi code thay đổi
body-parser	Xử lý dữ liệu gửi từ client
fs (hoặc lowdb)	Lưu dữ liệu vào file JSON đơn giản

CẤU TRÚC DỰ ÁN

bash

```
profile-api/  
├── data/  
│   └── profiles.json      # Dữ liệu mẫu
```

```
├── routes/
│   └── profiles.js          # Các route API
├── controllers/
│   └── profileController.js
├── app.js                  # Khởi tạo Express App
└── package.json
```

TÓM TẮT CÁC BƯỚC THỰC HIỆN

1. Khởi tạo dự án (npm init)
2. Cài đặt thư viện
3. Tạo file dữ liệu JSON
4. Tạo Express app (app.js)
5. Tạo route /profiles
6. Viết controller xử lý logic API
7. Kiểm thử bằng Postman hoặc gọi từ React Native app

CHI TIẾT BƯỚC THỰC HIỆN

1. Khởi tạo

bash

```
mkdir profile-api && cd profile-api
npm init -y
npm install express body-parser nodemon
```

Thêm vào package.json:

json

```
"scripts": {
  "dev": "nodemon app.js"
}
```

2. Tạo app.js

js

```
const express = require('express');
const bodyParser = require('body-parser');
const profileRoutes = require('./routes/profiles');

const app = express();
app.use(bodyParser.json());
app.use('/profiles', profileRoutes);

app.listen(3000, () => console.log('Server running on port 3000'));
```

3. Tạo file dữ liệu data/profiles.json

```
json

[
  {
    "id": 1,
    "name": "Nguyễn Văn A",
    "job": "Frontend Developer"
  },
  {
    "id": 2,
    "name": "Trần Thị B",
    "job": "UI/UX Designer"
  }
]
```

4. Tạo routes/profiles.js

```
js

const express = require('express');
const { getAll, getOne, addOne } =
  require('../controllers/profileController');
const router = express.Router();

router.get('/', getAll);
router.get('/:id', getOne);
router.post('/', addOne);

module.exports = router;
```

5. Tạo controllers/profileController.js

```
js

const fs = require('fs');
const path = './data/profiles.json';

const getProfiles = () => {
  const data = fs.readFileSync(path);
  return JSON.parse(data);
};

exports.getAll = (req, res) => {
  res.json(getProfiles());
};

exports.getOne = (req, res) => {
  const profiles = getProfiles();
  const profile = profiles.find(p => p.id == req.params.id);
  profile ? res.json(profile) : res.status(404).json({ error: 'Not found' });
};

exports.addOne = (req, res) => {
  const profiles = getProfiles();
  const newProfile = {
    id: profiles.length + 1,
    name: req.body.name,
```

```
    job: req.body.job
  };
  profiles.push(newProfile);
  fs.writeFileSync(path, JSON.stringify(profiles, null, 2));
  res.status(201).json(newProfile);
};
```

PHẦN 2: BÀI TẬP THỰC HÀNH

BÀI TẬP 1: Tạo Giao Diện Danh Sách Profile

- **Tên:** Giao diện danh sách bằng StyleSheet
- **Mục đích:** Ôn tập cách dùng StyleSheet và cấu trúc component
- **Tóm tắt:** Hiển thị danh sách người dùng từ API Node.js
- **Yêu cầu:** Gọi API /profiles, hiển thị thẻ name và job
- **Test cases:**
 1. Hiển thị đúng 2 profile từ API
 2. Không có lỗi khi API trả về rỗng
 3. Có thể scroll khi danh sách dài
- **Công nghệ:** React Native, StyleSheet, axios
- **Cấu trúc:**
 - o screens/HomeScreen.js
 - o components/ProfileCard.js
- **Bước thực hiện:**
 1. Gọi API từ useEffect
 2. Dùng FlatList để hiển thị
 3. Dùng StyleSheet để style
- **Kết quả đạt được:** Giao diện cơ bản có thể hiển thị dữ liệu từ backend

BÀI TẬP 2: Dàn Bố Cục Profile Bằng Flexbox

- **Tên:** Profile Layout Flex
- **Mục đích:** Rèn kỹ năng dùng flexDirection, justifyContent, alignItems
- **Tóm tắt:** Căn giữa avatar + thông tin profile, layout dạng hàng
- **Yêu cầu:** Layout phải responsive khi đổi chiều màn hình
- **Test cases:**
 1. Các item hiển thị theo hàng ngang
 2. Avatar nằm bên trái, thông tin bên phải
 3. Không vỡ layout khi kéo ngang
- **Chức năng:** Layout Flexbox
- **Công nghệ:** React Native, StyleSheet (Flexbox)
- **Cấu trúc:** Trong ProfileCard.js
- **Bước thực hiện:**
 1. Dùng flexDirection: 'row'
 2. Dùng justifyContent: 'space-between', alignItems: 'center'
- **Kết quả đạt được:** Thành thạo Flexbox trong layout profile

BÀI TẬP 3: Tạo Styled Screen

- **Tên:** Màn hình styled-components
- **Mục đích:** Sử dụng `styled-components` để tạo UI hiện đại
- **Tóm tắt:** Tạo màn hình dùng styled-components để hiển thị danh sách
- **Yêu cầu:** Không sử dụng `StyleSheet`, chỉ styled-components
- **Test cases:**
 1. Thay đổi màu nền component thông qua props
 2. Có thể reuse component (nhiều profile)
 3. Giao diện hiển thị tốt trên iOS và Android
- **Chức năng:** Styled UI
- **Công nghệ:** React Native, styled-components
- **Cấu trúc:**
 - `screens/StyledScreen.js`
- **Bước thực hiện:**
 1. Tạo các styled component (Container, Card, Avatar, Text)
 2. Map dữ liệu từ API
 3. Dùng props để custom color
- **Kết quả đạt được:** Giao diện styled hiện đại, thành thạo styled-components

TÓM TẮT KẾT QUẢ CẦN ĐẠT

Kiến thức	Thành tựu đạt được
StyleSheet	Biết cấu trúc và viết style tĩnh
Flexbox Layout	Hiểu rõ bố cục hàng/dọc, canh giữa
Styled-Components	Dùng được styled-components hiện đại
Backend API	Tạo được backend cung cấp dữ liệu
Kết nối Frontend/Backend	Hiển thị dữ liệu từ Node.js qua React Native